InsightDesk: LLM-Verified Grouping of Customer Complaints into Problem Cases for Incident Detection and Escalation

Krushnam Sadrani, Sanyogita Shinde, Krisha Sinnarkar
Department of Artificial Intelligence & Data Science, K. J. Somaiya Institute of Technology, Sion, Mumbai
Under the guidance of Prof. Ganesh Wadmare
Academic Year 2025–26

***Abstract*— *Support teams receive the same problem described in many different words. Counting, alerting and escalation all depend on first grouping these raw complaints into problem-level cases, yet published systems either cluster offline into topics, rely on per-ticket classification, or need structured telemetry. We present InsightDesk, an online pipeline that assigns each arriving complaint to a problem case: similarity retrieval over diverse case exemplars narrows each complaint to at most five candidate cases, a calibrated similarity band settles clear matches and clear non-matches, an LLM judge makes one bounded decision whenever similarity alone is not conclusive, and a periodic re-joining pass merges cases that online decisions split. On top of cases it computes per-case spike signals with a Poisson CUSUM, groups related cases for alerting, raises a separate alarm for vague complaints that fit no case, and escalates only after human approval. On 338 human-labelled Play Store reviews of six help-desk products, InsightDesk reaches pairwise precision 0.83, recall 0.35 and F1 0.49, against F1 0.05–0.25 for published app-review grouping and topic-model baselines on the same data. With the same upper threshold it reaches precision 0.87–0.96 on Bugzilla duplicate benchmarks (F1 0.78–0.79), case precision 0.92 on three unseen products and 0.89 on an unseen company. On outbreaks injected into real complaint streams, its CUSUM detects 76–91% on bug-report streams and 42–43% on short-review streams at under one false alarm per 100 windows, where the per-tag z-score rule common in feedback-analytics tools detects none, and outages made only of vague complaints are detected in all cases.*

Keywords*— *Complaint intelligence, duplicate detection, LLM-as-a-judge, sentence embeddings, online clustering, spike detection, Poisson CUSUM, sentiment analysis, BERTopic, root-cause analysis, human-in-the-loop.

I. INTRODUCTION

A B2B SaaS support desk receives a steady stream of complaints through several channels. The operational questions are problem-level: how many customers are hit by the same issue, is it suddenly growing, what is the likely cause, and who must act. Answering them requires a step that most tools skip or hide: deciding, as each complaint arrives, whether it reports a problem already known or a new one.

This step is hard for three reasons. First, complaints are short and noisy (a median of 21 words in our Play Store data, mean 28) and two reports of one problem often share almost no vocabulary. Second, the decision must be online and stable: a case that changes membership when the input order changes cannot carry counts, spike history or an escalation record. Third, a wrong merge is costly, because it inflates a case and can trigger a false escalation.

The contributions of this paper are:

- An online retrieve-then-verify grouping method for raw complaints: exemplar-based case retrieval, a similarity band calibrated from labelled pairs with a per-channel lower threshold (Section IV-C), and an LLM judge consulted only inside the band, followed by an LLM-verified case re-joining pass.

- Per-case incident signals built on those cases: a Poisson CUSUM spike detector on small counts and alert groups over related cases, with a human-approved escalation policy.
- A vague-complaint alarm: complaints too vague to join any case ("the app is not working") are counted per product and tested for a surge, so an outage that customers describe only vaguely still raises an alert. To our knowledge no published system monitors the complaints its grouping cannot place.
- An evaluation on human-labelled real complaints, independent Bugzilla duplicate labels, unseen products and an unseen company, with pre-declared acceptance rules and cluster-bootstrap confidence intervals whose coverage we verify by simulation.
- A head-to-head comparison against published grouping methods, topic models, embedding thresholds, a tuned cross-encoder pipeline and general-purpose LLMs on identical data.

Section II reviews related work. Section III states the problem and gives the architecture, and Section IV describes each phase of the methodology in detail. Section V covers the implementation, Section VI the data and evaluation protocol, and Section VII the results. Sections VIII and IX conclude and outline future work.

II. LITERATURE SURVEY

Work relevant to InsightDesk falls into four families, compared with their parameters in Table I.

A. App-Review Mining and Emerging-Issue Detection

Mining app reviews for software engineering is a mature field [1]. MERIT [2] tracks topics across app versions with an online joint sentiment-topic model and flags topics whose share rises in a new version, explaining them with extracted phrases. Earlier, CLAP [21] grouped reviews that report the same bug with DBSCAN over textual features for release planning. These systems work per version or per topic rather than per problem case, and none decides membership online.

B. Duplicate Detection and LLM Judges

A recent large study of duplicate bug-report detection [3] finds that classic retrieval such as REP [22] remains competitive with deep models, and Cupid [4] improves it by letting an LLM extract the essentials of each report first (Recall-Rate@10 0.60–0.65). Bi-encoders such as SBERT [23] give indexable embeddings, while cross-encoders built on BERT [24] are more accurate but cannot be indexed. LLMs used zero-shot as matchers reach F1 76.4–98.4 on six entity-matching benchmarks [5], and LLM judges agree with human preferences on many tasks [6], [7]; their known position and verbosity biases argue for bounding each decision, which InsightDesk does with retrieval and calibrated thresholds.

C. Near-Duplicate and Semantic Deduplication

SimHash [25] and MinHash-LSH [26] detect copies at web scale, and SemDeDup [8] removes pairs of embeddings closer than a small distance ε. Embedding benchmarks [9] show that no single model leads on every clustering task. These methods target near-copies, not paraphrases of the same problem.

D. Incident Management and Feedback Analytics

iPACK [10] links customer tickets to incidents using cloud alerts (F1 0.87–0.94), and TickIt [11] groups and escalates tickets with an LLM (F1 0.88). LLM-based root-cause recommendation for cloud incidents [12], [13] grounds its explanations in telemetry, in the spirit of retrieval-augmented generation [15]. Commercial voice-of-customer tools such as SentiSum [14] detect anomalies per tag or theme, typically with z-scores, and publish neither their grouping method nor its accuracy.

E. Research Gaps Addressed

Three gaps shape InsightDesk. First, existing systems group by topic, version or predefined category, not into problem cases decided online as complaints arrive, so they cannot carry a stable count per problem. Second, spike detection in feedback tools runs on per-tag z-scores, which are unstable at the zero-to-two complaints per window that a single problem produces, and nothing monitors the complaints that the grouping cannot place. Third, no published system we found combines online problem-case grouping of raw complaint text with per-case spike detection, cause explanation and human-approved escalation, and reports labelled accuracy for the grouping step.

TABLE I
COMPARATIVE LITERATURE SURVEY

Ref	Title (venue, year)	Parameters	Limitations	Our Improvement
[2]	MERIT (TSE, 2022)	Topic share per app version	Topics, not problems	Problem-level cases
[3]	Duplicate bug reports (TOSEM, 2023)	BM25F / deep retrieval, top-k	Ranking only, no cases	Online cases with counts
[4]	Cupid (arXiv, 2023)	LLM summary + BM25F	No grouping decision	Calibrated band + LLM judge
[5]	LLM entity matching (EDBT, 2025)	Zero-shot prompt per pair	Pairwise, costly	One judge call per complaint
[8]	SemDeDup (arXiv, 2023)	ε = 0.00095–0.03	Near-copies only	Paraphrase-level matching
[10]	iPACK (ICSE, 2023)	Ticket–alert linking	Needs cloud alerts	Raw complaint text only
[11]	TickIt (FSE, 2025)	Predefined categories	Cannot open new problems	Open-ended problem cases
[13]	RCACopilot (EuroSys, 2024)	Telemetry + LLM	Needs telemetry	Root cause from complaints
[14]	SentiSum (2025)	z-score per tag, ~100-day baseline	Fails at 0–2 per window	Poisson CUSUM + alert groups

III. PROPOSED SYSTEM

A. Problem Statement

Help-desk and customer-support platforms such as Vision Helpdesk, Zendesk, Freshdesk, Intercom and ServiceNow, and the companies that use them, receive complaints from app reviews, support tickets, chat and web forms. The same problem is reported many times in different words, so agents triage it again and again, and no one sees how many customers a single problem affects. Existing tools group complaints by theme ("login") rather than by problem ("the OTP never arrives"): keyword rules and topic models mix different problems, and a single similarity threshold either merges unrelated complaints or misses paraphrases. Feedback tools flag spikes with z-scores on whole categories, which miss a new problem growing from a low base. There is a need for a system that groups raw complaints into problem-level cases as they arrive with high precision, tracks each case's volume to raise incidents early, adds sentiment, topic and root cause, and routes each incident to the responsible department after a person approves it.

B. System Architecture Overview

InsightDesk runs nine phases in four stages: Fig. 1 shows the flowchart of the pipeline and Fig. 2 its system architecture. In the first stage, intake and grouping (phases 1–4), each complaint is cleaned, embedded and either joined to an existing case, opened as a new case, or set aside as too vague to place. In the second stage, case upkeep (phases 5–6), a re-joining pass every 6 hours merges cases that describe one problem, and each case receives a sentiment label, a topic and the department responsible for that topic. In the third stage, incident signals (phases 7–8), complaint counts per case in 6-hour windows feed a Poisson CUSUM spike detector, and vague complaints feed a separate per-product alarm. In the fourth stage, action (phase 9), an eligible case receives an LLM-written root-cause explanation, and an email

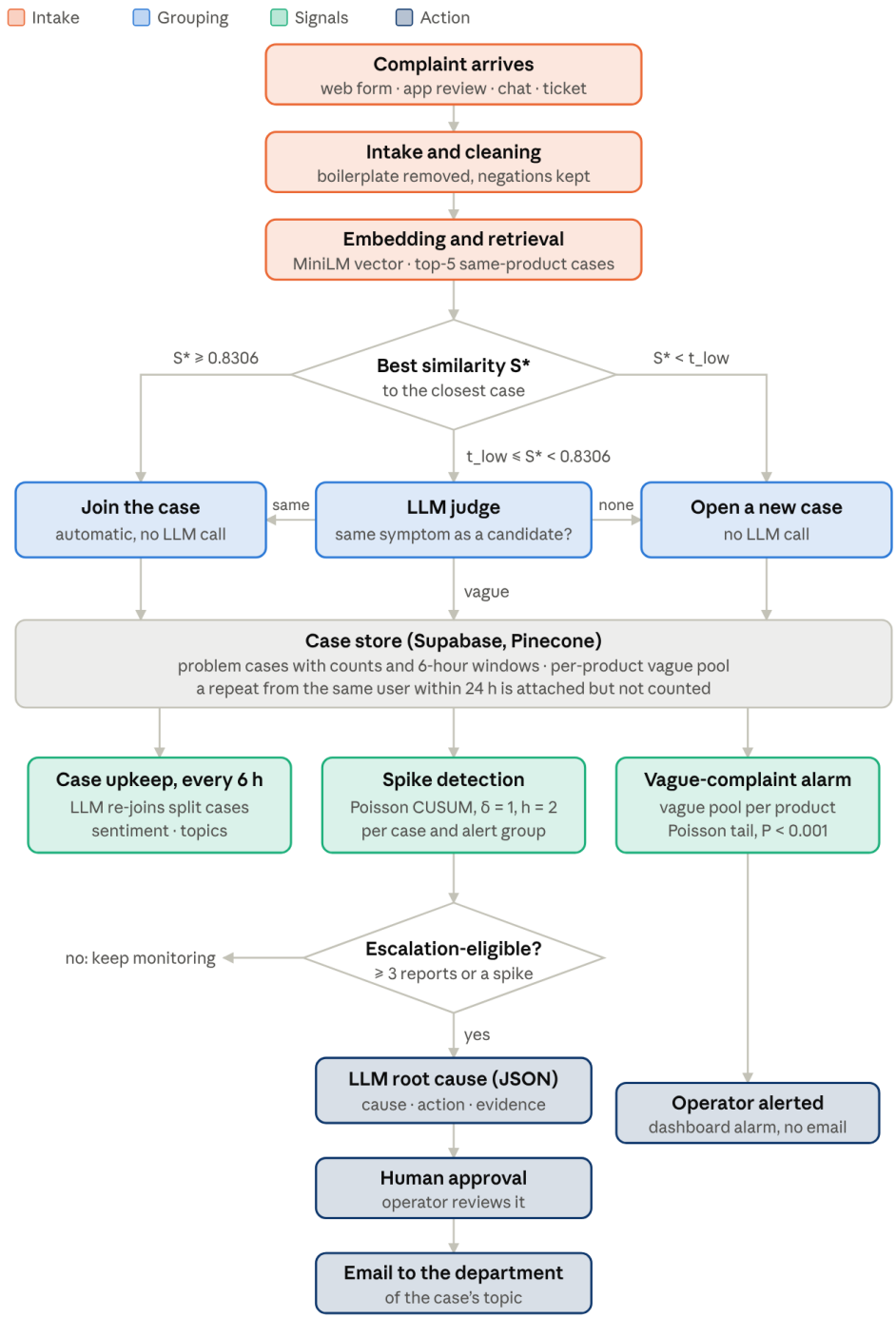


Fig. 1: Flowchart of the InsightDesk pipeline (t_low = 0.40 for app reviews and chat, 0.26 for tickets and bug reports)

reaches the department only after a person approves it. Cases carry counts, a spike history and an escalation record in Supabase (PostgreSQL); case exemplar vectors live in Pinecone. Sentiment and topic labels are attached to cases for the dashboard and routing and are not used in the grouping decisions.

IV. METHODOLOGY

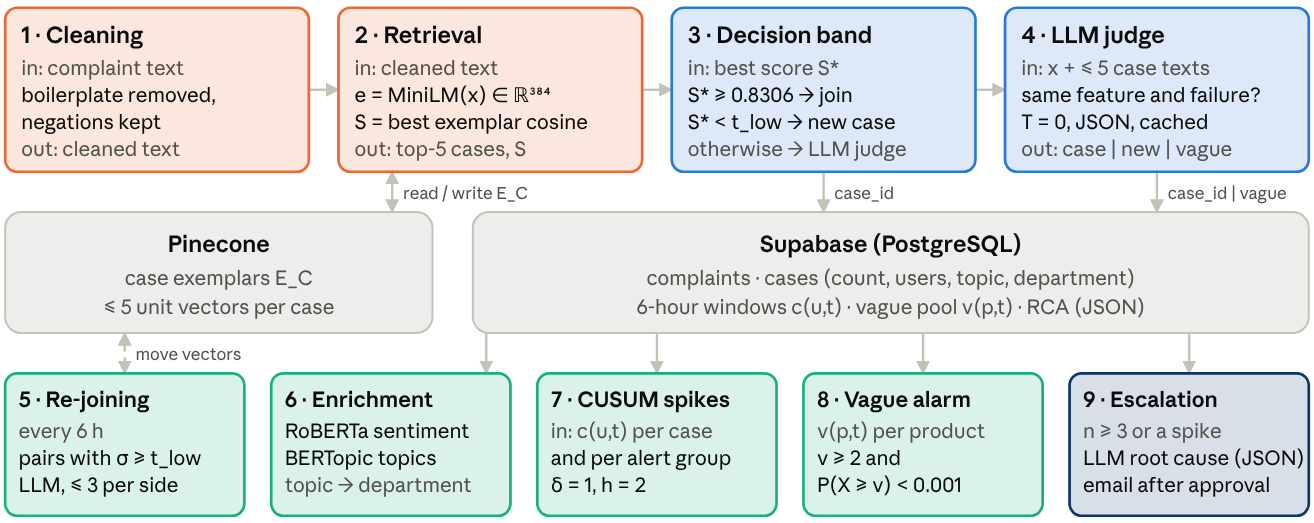


Fig. 2: System architecture: nine phases over two stores

This section describes each phase in detail: its input, the rule it applies with its parameters, and its output. Fig. 1 shows the path of a complaint through these phases, and Equations (1)–(11) give each rule. Throughout, a running example follows the complaint "Can't log in since the update, the OTP never arrives".

A. Phase 1 – Intake and Cleaning

A signed-in customer submits a complaint through the React dashboard or an API client; the FastAPI service stores it with its product, user and source channel (for example web form, app review or chat). Text beyond 5,000 characters is truncated. Cleaning uses label-free boilerplate suppression, recomputed every 6 hours over the most recent stored complaints (at most 2,000). The text is split into segments (sentences, clauses and a lead-in phrase). A segment is a boilerplate candidate when it recurs in at least 8 complaints with segment cosine similarity of at least 0.90, and it is confirmed as boilerplate when the remaining text of those complaints is unrelated (mean cosine below 0.30), as with greetings such as "Hi team, please help". A segment of an arriving complaint that is a near-copy (cosine at least 0.90) of a listed boilerplate segment is dropped, while at least one segment is always kept; on our Play Store reviews the rule finds no boilerplate, while on Bugzilla reports it removes the bug-template lines; the text is lower-cased and stripped of punctuation and stop words. Negations such as not, never and no are kept, because "works" and "doesn't work" must not look alike. The original text is kept for display and for the LLM judge. For the running example, only the complaint itself remains.

B. Phase 2 – Embedding and Case Retrieval

Let x be a complaint and e(x) the L2-normalised MiniLM [19] embedding of its cleaned text (384 dimensions), so similarity is the dot product s(x, y) = e(x)·e(y). Each case C keeps a set E_C of at most five exemplars. A new member m of C is added to E_C only if it is not already represented:

$$\max_{m' \in E_C} s(m, m') < 0.85, \qquad |E_C| \leq 5 \tag{1}$$

The score of complaint x against case C is its best match among the exemplars, and only cases of the same product are considered:

$$S(x, C) = \max_{m \in E_C} s(x, m), \qquad C_5(x) = \text{top-5 cases of product}(x) \tag{2}$$

Exemplars let one case match differently worded reports of the same problem, which a single centroid cannot. Each exemplar is stored in Pinecone with its case, product, vague flag and the text of the complaint that founded the case; one query returns the 60 nearest vectors, which are ranked per case by their best exemplar. For the running example, suppose the closest case is "OTP not received on login" with S* = 0.71 (an illustrative value).

C. Phase 3 – Calibrated Decision Band

For the best candidate C* with score S* = S(x, C*), the decision is

$$d(x) = \text{join } C^* \text{ if } S^* \geq t_{high} \quad (3)$$

$$d(x) = J(x, C_5(x)) \text{ if } t_{low} \leq S^* < t_{high}$$

$$d(x) = \text{new case if } S^* < t_{low}$$

where J is the LLM judge (Phase 4). Both thresholds are computed from labelled pairs within the same product: in every Play Store evaluation from the tuning half only, and for the deployed system once from all labelled Play Store reviews. With y_i the true problem of complaint i and s_ij the similarity of complaints i and j:

$$\text{Prec}(t) = \frac{|\{(i,j) : s_{ij} \geq t,\ y_i = y_j\}|}{|\{(i,j) : s_{ij} \geq t\}|} \quad (4)$$

$$t_{high} = \min\{\, t : \text{Prec}(t) \geq 0.97 \,\}$$

so that automatic joins are at least 97% correct on tuning data, and

$$t_{low} = Q_{0.20}(\{\, s_{ij} : y_i = y_j \,\}) \quad (5)$$

the 20th percentile of duplicate-pair similarities, so that 80% of true duplicate pairs lie at or above the band's lower edge. Calibrated on all 338 labelled Play Store reviews this gives t_low = 0.26 and t_high = 0.8306, the values used in the deployed system and in every test on other data (Bugzilla, unseen products, the unseen company, IT tickets); within the Play Store halves each tuning half yields its own values (t_high 0.70–0.77, t_low 0.24–0.30), so no Play Store score uses thresholds that saw the scored reviews. The 20th-percentile rule proved conservative on short text: raising the lower threshold to 0.40 on app reviews improved precision and recall together while sending fewer complaints to the judge, and the setting was confirmed on unseen apps under a pre-declared rule, whereas on long bug reports it made no real difference (Section VII-B). The deployed system therefore uses a per-channel lower threshold and one upper threshold for every channel:

$$t_{low}(x) = 0.40 \text{ if channel}(x) \in \{\text{app review, chat}\} \quad (6)$$

$$t_{low}(x) = 0.26 \text{ otherwise (tickets, bug reports, web form)}$$

In the running example S* = 0.71 lies inside the band, so the judge decides.

D. Phase 4 – LLM Judge

Inside the band an LLM (gpt-oss-120b [20]) receives the complaint and the founding text of each candidate case and answers which case reports the same underlying problem, "none", or "too vague to tell". A gate in the prompt asks for a specific shared symptom rather than a shared topic: the same feature and the same failure, so that one fix would solve both. The judge runs at temperature 0 with a JSON answer, and every decision is cached. A chosen case is joined and its count rises by one; "none" opens a new case; "too vague" places the complaint in the unassigned pool of its product (Phase 8); an API error, timeout or malformed answer opens a new case, so a failure never merges. In the running example, "the OTP never arrives" matches the symptom of "OTP not received on login", so the complaint joins that case; a complaint that "login is slow" would open a new one.

The judge never acts alone. Retrieval decides which cases it may choose from (at most five, same product only), the thresholds settle clear matches and clear non-matches without it, a failed or uncertain answer never merges, re-joining limits each case to one merge per pass, and no escalation leaves the system without human approval. Each complaint receives at most one judge decision, never pairwise comparisons. In our runs the band covered 66% of short reviews at the deployed setting and 90–96% of bug reports. The judge answered "too vague to tell" for 20% of Play Store complaints and 7% of bug reports (at t_low 0.26). A decision takes 1.34 s at the median and costs about $0.24 per 1,000 judged complaints (Section VII-E).

E. Phase 5 – Case Re-joining

Online decisions split some problems across several cases. Every 6 hours a scheduler inside the API runs a re-joining pass. For every case A it considers its three nearest same-product cases B whose case similarity reaches t_low:

$$\sigma(A,B) = \max_{a \in A,\ b \in B} s(a,b) \geq t_{low} \qquad (7)$$

Candidate pairs are examined in decreasing σ. An LLM case-pair judge sees up to three complaints per side and decides whether both cases describe one problem. Each case takes part in at most one merge per pass, which keeps a single wrong decision from cascading. On a merge, counts are summed and the complaints, 6-hour windows and exemplar vectors of the absorbed case move to the kept case, which keeps its founding text; the absorbed case is hidden and is never offered to the judge again. On the Play Store held-out halves at the deployed setting, the judge approved 35 merges, of which 23 (66%) joined cases with the same majority problem; on Firefox and Thunderbird it approved 22 and 18 merges, of which 91% and 94% were correct. Re-joining is thus reliable on bug reports and noisier on short reviews.

F. Phase 6 – Enrichment: Sentiment, Topics and Routing

Each case receives a sentiment label from RoBERTa [17] (cardiffnlp twitter-roberta-base-sentiment-latest) with three classes (negative, neutral, positive) and its confidence, computed once from the case's founding complaint (its representative text), so later reports do not change it. Evaluated on real Play Store reviews against star ratings (1–2 negative, 3 neutral, 4–5 positive), it reaches accuracy 0.707, with precision 0.81 and recall 0.85 on negative complaints; DistilBERT reached accuracy 0.713 but is binary, cannot say neutral, and is less precise on negatives (0.77). Topics come from BERTopic [16] over case texts: UMAP [27] with 12 neighbours, 5 components and cosine distance, then HDBSCAN [28] with minimum cluster size 3 and minimum samples 1, and keywords from a 1–2-gram c-TF-IDF (minimum document frequency 2). Topics are recomputed on the first run, when the number of cases has grown by 20%, when outliers exceed 15%, or at a scheduled refresh; the 20% and 15% triggers are operating defaults chosen empirically, not tuned. Departments come from a fixed rule table, not an LLM: five departments (Engineering, Authentication, Payments, Customer Success, Platform) each have a list of trigger words (for Authentication: login, password, otp, signin, verification, token), a topic goes to the department whose list matches most of its five top keywords, a topic with no match goes to General Support, and an operator can override the department in the dashboard. On the 306 cases of the test deployment BERTopic found 44 topics and left 41 cases (13%) as outliers. Their c_v coherence is 0.49, against 0.38 for random five-word sets from the same texts, so the topics are moderately coherent; the largest are login and sign-in (19

cases), notifications (18), reinstalling or the app not working (16), support plans (15) and email addresses (11). The rule table routed only 19 of the 44 topics to a named department and sent 25 to General Support, so routing is the weakest step; a learned or LLM-based mapping is left to future work. In the running example the case is labelled negative and placed under an Authentication topic.

G. Phase 7 – Spike Detection

Complaints are counted per case u in 6-hour windows t, written c_ut, with zeros filled in for empty windows. A repeat complaint from the same user about the same case within 24 hours is attached to the case but not counted again. The expected rate is a smoothed mean over the previous L = 28 windows (seven days), where n_ut is the number of past windows available:

$$\lambda_{u,t} = \frac{\sum_{\tau=t-L}^{t-1} c_{u,\tau} + a}{n_{u,t} + b}, \quad L = 28,\ a = 0.5,\ b = 7 \tag{8}$$

A Poisson CUSUM [29] accumulates evidence that the count has risen by δ = 1 report per window above that rate, and raises a spike when the score reaches h (the score restarts at zero after each alarm):

$$S_{u,t} = \max(0,\ S_{u,t-1} + c_{u,t} - k_{u,t}), \quad k_{u,t} = \frac{\delta}{\ln((\lambda_{u,t} + \delta)/\lambda_{u,t})} \tag{9}$$

$$\text{spike if } S_{u,t} \geq h, \quad \delta = 1,\ h = 2$$

The prior (a, b) keeps new and quiet cases from alerting on a single complaint. Because one problem can still be split across cases, similar same-product cases are summed into alert groups and tested the same way, while the cases themselves stay separate. In the running example, 5 OTP complaints in one window against a usual rate of 0.1 raise a spike.

H. Phase 8 – Vague-Complaint Alarm

Vague complaints join no case, so no per-case detector can see them. They are counted per product p in the same windows, v_pt, with a baseline λ_pt computed as in Phase 7, and tested with a Poisson tail rule:

$$\text{alarm}(p,t) \Leftrightarrow v_{p,t} \geq 2 \ \wedge\ P(X \geq v_{p,t}) < 0.001, \quad X \sim \text{Poisson}(\lambda_{p,t}) \tag{10}$$

An alarm reports when the surge started and which cases were active in the same windows, so an outage reported only as “the app is not working” still reaches an operator. The threshold was fixed before testing, for the per-window Poisson test that the vague alarm reuses: among p ∈ {0.01, 0.001} and a minimum count m ∈ {2, 3, 4}, the pre-declared rule picked the setting with the highest detection whose false alerts stayed at or below 2 per 100 windows on both tuning sets, which was m = 2 with p < 0.001 (56% detection, at most 1.36 false alerts; p = 0.01 gave 3.35 on bug reports). Confirmed unchanged on 588 reviews of unseen apps, the alarm detected every 10-complaint vague outage at a cost of 0.19 extra false alerts per 100 windows.

I. Phase 9 – Escalation and Root-Cause Explanation

A case is eligible for escalation when it is not vague and has at least three counted reports or an active spike of the case or its alert group:

$$\text{eligible}(C) \Leftrightarrow \neg\text{vague}(C) \wedge (n_C \geq 3 \vee \text{alert}(C) \vee \text{alert}(\text{group}(C))) \tag{11}$$

For an eligible case an LLM produces a grounded cause hypothesis as JSON with the fields cause, affected segment, action, severity, confidence and evidence, restricted to evidence in the case’s complaints and spike history. An operator reviews it in the dashboard, and an email reaches the department of the case’s topic only after the operator approves it.

V. IMPLEMENTATION DETAILS

A. Software Stack

The backend is a Python FastAPI service. Complaints, cases, 6-hour window counts (each

case's spike history), root-cause reports and approved escalations are stored in Supabase (PostgreSQL), and case exemplar vectors in a Pinecone index. Embeddings come from sentence-transformers all-MiniLM-L6-v2. The LLM judge, the case-pair judge and the root-cause writer call gpt-oss-120b through the Groq API in JSON mode at temperature 0; a small local model can be configured instead. Sentiment uses the Hugging Face transformers library, and topics use BERTopic with UMAP and HDBSCAN. A scheduler inside the API runs re-joining and topic upkeep every 6 hours. The dashboard is a React and TypeScript application that shows cases, incidents, alert groups, the vague-complaint alarm and root causes, and approved escalations are sent by SMTP. The pipeline is covered by unit tests and an end-to-end test of the API.

B. Key Design Properties

- Bounded LLM use: the judge sees at most five same-product candidates, decides only inside the calibrated band, and makes one decision per complaint.
- Safe failure: a failed or uncertain judge answer opens a new case and never merges.
- Stable online cases: a decision, once made, is changed only by a logged re-joining merge, so case ids persist and cases can carry counts, spike history and an escalation record.
- Human approval: no escalation email leaves the system without an operator's approval.
- Reproducibility: every LLM decision is cached at temperature 0, so every reported run can be replayed.

VI. DATASETS AND EVALUATION PROTOCOL

A. Datasets

- Play Store reviews: 338 labelled reviews of six help-desk products (Zendesk, Freshdesk, Help Scout, Zoho Desk, LiveAgent, HappyFox), from a pool of 450 labelled by three team members. Labels follow a written codebook; inter-annotator agreement is Fleiss' $\kappa = 0.74$ after a re-check round (0.57 in the first round).
- Unseen products: 588 reviews of Intercom, Jira Service Management and ServiceNow, never used for tuning.
- Bugzilla: Firefox (three samples) and Thunderbird reports, with the project's own duplicate links as independent labels.
- Retrieval at scale: 233,521 Firefox, Thunderbird and Eclipse reports with the public BugHub split.
- IT tickets: 1,000 Endava support tickets with category labels.

B. Credibility of the Labels

Each dataset's ground truth comes from a different source, so no single labelling choice drives the results.

- Bugzilla (independent, maintained by the projects). Duplicate links are created by Mozilla's own triagers and developers during normal triage: a report is closed as RESOLVED DUPLICATE and linked to the report it repeats, after a person has compared the two. The labels predate this work, were made for engineering rather than research purposes, and the same Bugzilla histories are standard benchmarks for duplicate bug-report detection, including the public BugHub split used here. Because triagers link only the duplicates they notice, some true duplicates are unlinked; a correct merge of two unlinked duplicates is then scored as an error, so measured precision on Bugzilla is, if anything, conservative. We did not measure how many duplicates are unlinked in our samples, so we make no claim about the size of this effect.
- Play Store (human, three annotators). Three team members labelled the reviews against a written

codebook that defines a problem by its user-visible symptom (for example, “cannot log in” rather than “login”). On a 50-review overlap labelled by all three, a first round reached Fleiss’ $\kappa = 0.57$; after the three re-checked the codebook rules they had read differently, agreement rose to $\kappa = 0.74$ (95% CI 0.63–0.84), conventionally read as substantial; remaining disagreements were resolved by discussion, and a blind relabelling round checked consistency. As an external check, GPT, Claude and gpt-oss given the same codebook agree with the human key at $\kappa = 0.68$–0.70, about as well as the annotators agree with each other, which indicates the codebook is reproducible rather than idiosyncratic. The labels were fixed before any of the pipeline variants reported here was scored, and thresholds are always tuned and scored on disjoint halves.

- Unseen products. Labels for Intercom, Jira Service Management and ServiceNow are the agreed key of two independent LLM labellers using the same codebook; we treat them as secondary evidence alongside the human and Bugzilla labels. The two labellers agree only moderately (pairwise $\kappa = 0.38$); the key keeps a pair as a duplicate only when both grouped it, so it is conservative but may still reflect shared LLM biases. A human spot-check of 50 of these review pairs has been prepared but not yet completed.
- Endava IT tickets. Real IT service-desk tickets from a third domain: 1,000 tickets labelled only with one of 127 category codes. A category groups many different problems, so these labels test case purity, not duplicate detection, and are reported as a third-domain check only.
- Synthetic complaints were used only to check that each component behaves as designed and are not part of any reported comparison.

C. Protocol

Thresholds are tuned on one half of the problems and scored on the other half; we report the pooled scores of three held-out halves. Metrics are pairwise precision, recall and F1 over complaint pairs, B-cubed scores, and case precision (B-cubed precision: for each complaint, the share of its case that reports the same problem, averaged over complaints). Every method has a threshold that trades precision against recall, so neither number alone can rank methods: any method reaches high precision by merging almost nothing, or high recall by merging almost everything. We therefore compare methods in two ways. First, each baseline’s threshold is tuned for its best F1 on the tuning half, the setting most favourable to it. Second, because a wrong merge inflates counts and can trigger a false escalation while a missed duplicate only leaves two smaller cases, we also compare recall at equal precision, with every baseline threshold set to reach precision 0.80 on the tuning half. Confidence intervals come from a cluster bootstrap over problems in which pairs between two draws of the same problem are not counted; a simulation with known ground truth confirms 92–99% coverage for the nominal 95% intervals. Every pipeline change was compared against the current version on identical data under a pass rule written before the run.

D. Baselines

We compare against the methods other systems use for each step, run on identical data and scored with the same metric. For grouping: CLAP’s density clustering of app reviews, BERTopic topic grouping (the approach behind several commercial feedback tools), a single embedding-similarity threshold, a tuned cross-encoder threshold pipeline, and general-purpose LLMs used either as labellers or to group a whole batch in one call. For spike detection: the z-score rule that commercial feedback-analytics tools apply per tag or theme, which alerts when a window’s count exceeds its

mean by more than two standard deviations, applied to the same per-case counts. Topic models and clustering methods were not designed for problem-level grouping; we include them because commercial tools use them for this purpose, and score every method on the same task: placing reports of the same problem together and different problems apart. Each method's output is read as a partition (a topic, cluster or case per complaint) and scored pairwise against the same labels; each baseline is reported at the better of its published and tuned settings.

VII. RESULTS AND DISCUSSION

A. Comparison with Published Methods and LLMs

TABLE II
COMPETITORS EVALUATED ON OUR DATA

Method	Parameters	Limitation	Our Improvement
CLAP [21]	eps 0.6 / 0.75	Shared words merge problems	LLM judge separates them
BM25, REP [22]	k1 1.2, b 0.75, cut 0.15–0.20	Misses paraphrases	Semantic retrieval
MinHash-LSH [26]	3-char shingles, J ≥ 0.15	Near-copies only	Matches meaning
SimHash [25]	36–42 of 64 bits	Near-copies only	Matches meaning
BERTopic [16]	Library defaults	Themes, not problems	Symptom-level cases
Embedding cut [23]	One cosine threshold	Unsafe or too narrow	Two cuts + judge
Cross-encoder [24]	Score cut 0.4834	Fails in a stream	Stream-calibrated band
LLM labellers	Codebook or own problem list	Codebook knew the test problems; own list F1 0.32	Online, no codebook
LLM batch grouping	One prompt per batch	Order-sensitive (89% of pairs kept); no memory	Stable cases
z-score rule [14]	z > 2 per tag	Unstable at small counts	Poisson CUSUM

InsightDesk has the highest F1 of all methods that keep stable online cases: 0.49 on Play Store, with precision 0.83 and recall 0.35. Published duplicate finders given the same input, the same online procedure and their most favourable threshold reach F1 0.12–0.24; with their threshold chosen to reach precision 0.80 on the tuning half they recover about 1% of duplicate pairs on the held-out half, while InsightDesk recovers 35% at precision 0.83. Word and character overlap cannot separate different problems that share vocabulary, which is the case the LLM judge exists for. Fig. 4 places every method at its best-F1 setting. Topic models group by theme rather than by problem, and BERTopic's precision of 0.16 shows how many unrelated complaints share a topic. General-purpose LLMs that label each review with our codebook group better offline (precision 0.83–0.87, F1 0.64–0.67), but the codebook was written from these same reviews and already names the test problems; when the same model must build its own problem list from earlier reviews of the same companies and then classify new ones, precision falls to 0.62 and F1 to 0.32, below InsightDesk on the same halves (0.81 and 0.39 at t_low 0.26). A single LLM call that groups a whole batch of one product reaches a similar offline F1 (0.50), but its groups depend on the order of the input: listing the same reviews in three shuffled orders kept 78–95% (mean 89%) of its grouped pairs, against 99% for an identical rerun, and a batch call has no memory of earlier batches, so it cannot keep case ids, counts or spike histories as complaints arrive. CLAP's published setting reaches a similar precision (0.84) but finds only 2% of duplicate pairs.

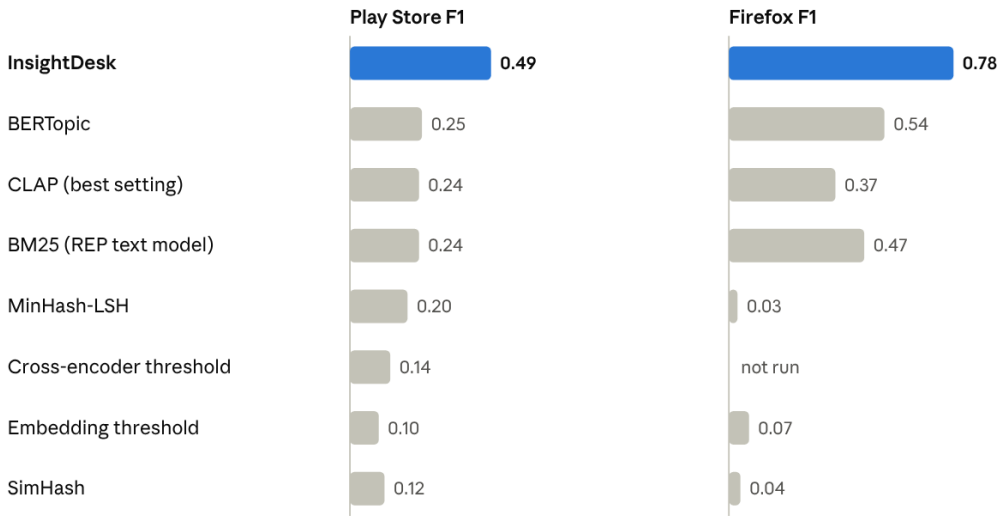


Fig. 3: Pairwise F1 against published methods on identical data (Play Store held-out halves; Firefox sample 3)

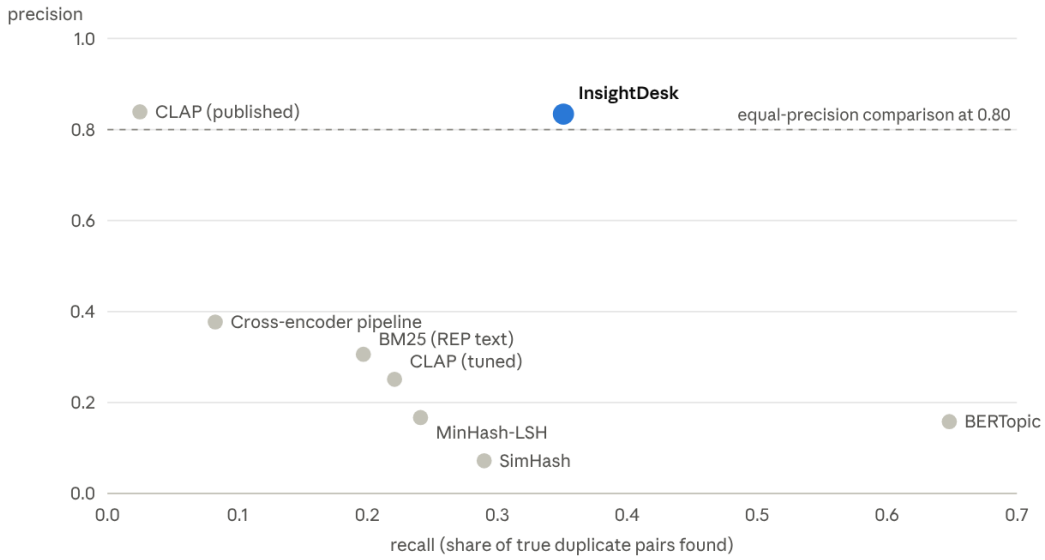


Fig. 4: Pairwise precision and recall of every method, Play Store held-out halves

Re-joining recovers duplicates that online decisions split, on short reviews and long bug reports alike (Fig. 5). On the Play Store held-out halves at the deployed setting it raises recall from 0.284 to 0.351 (+0.067, 95% CI +0.032 to +0.110) and F1 from 0.430 to 0.494 (+0.064, CI +0.025 to +0.112), at a cost in precision from 0.883 to 0.834 (CI −0.139 to +0.002). On bug reports precision is unchanged while recall rises by 11 points. Without re-joining, the grouping step alone therefore reaches precision 0.88 on short reviews; the deployed system trades five points of it for a quarter more duplicates found.

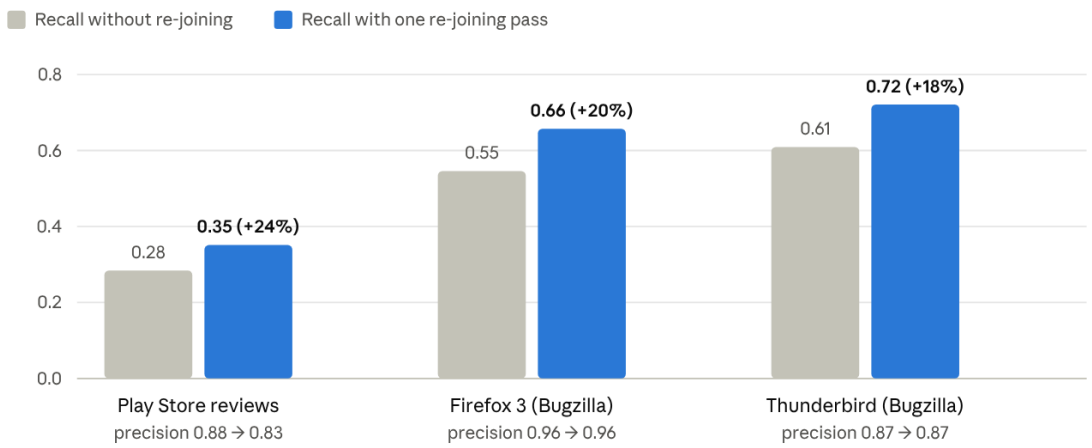


Fig. 5: Recall with and without one re-joining pass; precision shown below each pair

B. Generalisation

TABLE III
RESULTS ON UNSEEN DATA (UPPER THRESHOLD FIXED)

Setting	Data	Precision	Note
Second domain	Firefox, 3 samples	0.92–0.96	F1 0.78 with re-joining
Second domain	Thunderbird	0.87	F1 0.79 with re-joining
Unseen products	Intercom, Jira SM, ServiceNow	case 0.92	No tuning on them
Unseen company	Leave-one-company-out	case 0.89	Other companies' thresholds
No product field	889 bug reports, one stream	0.90	vs 0.91 with product
Retrieval at scale	233,521 bug reports	RR@5 0.46–0.56	BM25 0.40–0.44, REP 0.43–0.44
Third domain	1,000 Endava tickets	case 0.90	Cross-encoder 0.66

The same upper threshold transfers from short app reviews to long bug reports, and to products and a company never seen in tuning. This transfer is also the main evidence that the rules of Section IV-C (the 0.97 precision target for t_high and the 20th percentile for t_low) give a robust operating point rather than one fitted to the tuning data. A sensitivity check supports this for t_high: around each half's calibrated thresholds, lowering t_high by 0.03 or 0.08 keeps precision at 0.81–0.83 and F1 at 0.36–0.41 (calibrated setting: 0.81 and 0.39), while lowering it by 0.13 drops precision to 0.72. Only 4% of held-out Play Store complaints (20 of 505) reached an automatic join above t_high, against 86% decided by the judge, so on short reviews the judge does the deciding.

Raising t_low, measured with live judge calls, leaves recall unchanged (0.256–0.264) while precision rises to 0.87 at +0.09 and 0.91 at +0.14 and the share of complaints sent to the judge falls from 86% to 61%. The 20th-percentile rule is therefore conservative on short reviews. Under a pre-declared confirmation on Firefox and Thunderbird, however, t_low = 0.40 left precision (0.912 vs 0.908) and recall (0.571 vs 0.578) essentially unchanged and reduced the judged share only from 96% to 90%, below the 10-point bar, because similarities of long bug reports mostly lie well above 0.40. A second pre-declared check on unseen short text, the 588 reviews of three unseen apps, met its bar: with t_low = 0.40 the judged share fell from 94% to 77%, precision rose from 0.608 to 0.635 and recall changed from 0.284 to 0.255, within the allowed 0.03. On the Play Store held-out halves (each half with its own t_high), t_low = 0.40 gives precision 0.883, recall 0.284 and F1 0.430 before re-joining, against 0.810, 0.259 and 0.392, with 66% instead of 86%

of complaints judged. InsightDesk therefore uses t_low = 0.40 for short-text channels (app reviews, chat) and keeps 0.26 for tickets, bug reports and its own complaint form; the unseen-app labels are LLM-made, so this setting is supported by secondary evidence. Recall is higher on bug reports, whose descriptions are specific, than on short reviews (median 21 words), where reports of one problem often share little wording.

Complaint text varies by channel: an app review (median 21 words) and a bug report (median 87–92 words) describe problems very differently. When every method's threshold is tuned on Play Store reviews and applied unchanged to Firefox reports, InsightDesk's precision rises from 0.83 to 0.96, while surface-similarity methods break down: MinHash-LSH and SimHash merge almost every pair (precision 0.01–0.02), and BM25 falls to precision 0.56 (Fig. 3). Their scores depend on text length and shared vocabulary, so a cut tuned on short reviews admits nearly all pairs of long reports. InsightDesk's thresholds act on length-insensitive embedding similarity and only settle clear cases; uncertain pairs are decided by an LLM that reads both texts. At their published or default settings, which were not tuned on short reviews, the baselines do better on Firefox (CLAP F1 0.37, BERTopic 0.54) but remain below InsightDesk's 0.78, and choosing their setting for a new format needs labelled data in that format. In practice a new deployment can start from the shipped calibration with no labelled data in its own format. Pairwise precision on unseen help-desk products (0.61–0.64) is lower than on the calibration data, so a small calibration set, or corrections that operators already make in the dashboard, would further improve a deployment. On 1,000 Endava IT service tickets, with the same settings, 90% of InsightDesk's cases fall within a single ticket category (case precision 0.904 [0.896, 0.947]), against 0.655 for the earlier cross-encoder pipeline, while it groups far fewer tickets, as category labels are coarser than problems.

C. Incident Detection

Outbreaks were simulated by moving the real reviews of one problem into a short burst inside a 30-day stream of 6-hour windows. On unseen Firefox and Thunderbird streams, the CUSUM detects 76% of outbreaks per case and 91% per alert group, at 0.38 and 0.69 false alerts per 100 windows; on short Play Store reviews, where outbreaks split across more cases, it detects 42–43% at 0.06. The z-score rule used by commercial feedback tools detects none on the same counts: with zero to two complaints per window a standard deviation gives no stable baseline. We compared the standard small-count alternatives on the same simulations. A raw count rule either over-alerts (at least 2 reports: 0.72–4.33 false alerts per 100 windows) or misses most outbreaks (at least 3: 25–49% detected), so a model of the baseline rate is needed. A Poisson tail test on each window (count ≥ 2, $P < 0.001$) detected 71% and 84% on the unseen streams at 1.20 and 1.75 false alerts, three times the CUSUM's, which is why InsightDesk uses the CUSUM. An EWMA chart on the same baseline (α 0.2–0.3, L 2–3) reached 86% detection on bug reports but at 3.2–3.6 false alerts per 100 windows, about nine times the CUSUM's. Across $h \in \{1, 1.5, 2, 3, 4\}$ and $\delta \in \{0.5, 1, 2\}$, $h = 2$ with $\delta = 1$ is the knee: $h = 1.5$ adds 10 points of detection for ten times the false alarms, and $h = 3$ loses a third of detections; $h = 2$ was chosen on Play Store and a second Firefox sample and confirmed unchanged on the unseen streams under a pre-declared rule. Real complaint counts are overdispersed: on 17,236 Firefox reports from the last two years, per-component variance-to-mean ratios have median 1.60 and pooled value 2.37 (95% CI 1.70–3.30). Nominal Poisson probabilities therefore understate false alarms (the

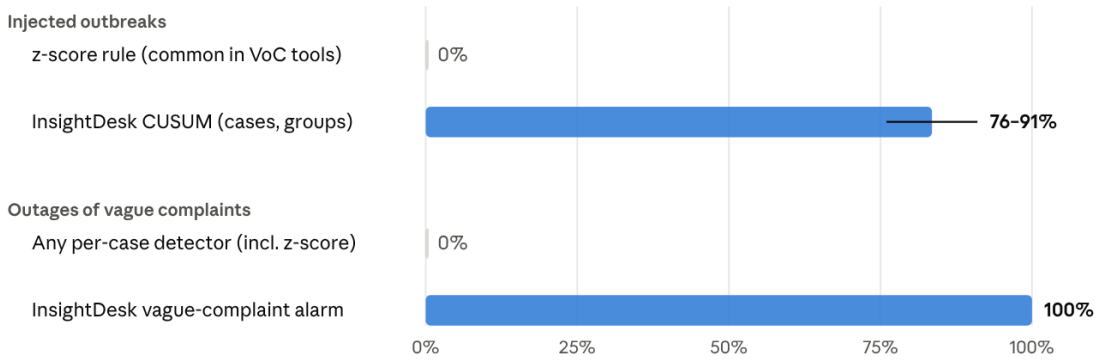


Fig. 6: Share of simulated incidents detected in 6-hour windows; InsightDesk under one false alarm per 100 windows

tail test fires 0.61 times per 100 real windows against 0.10 nominal), so all rates above are empirical. Because the CUSUM looks for a rise of one report per window, it is relatively more sensitive on very busy cases; this regime was rare in our streams. Two blind LLM raters preferred InsightDesk's grounded cause-explanation prompt over an alternative prompt (12:0 and 17:2) and found fewer unsupported claims.

D. Vague-Complaint Alarm

About one review in five is too vague to join any case (Phase 4), so an outage that customers describe only as "the app is not working" is invisible to every per-case detector: they detected none of the simulated vague-only outages. Counting these complaints per product and testing for a surge detected every vague-only outage of 10 or more complaints and 52% of those with 5, over 300 simulated outages per size, while false alerts rose from 0.20–0.24 to 0.35–0.39 per 100 windows. The result held on a second dataset of unseen help-desk apps.

E. Limitations

- Recall on short reviews. Pairwise recall on Play Store is 0.35. Short reviews of one problem often look no more alike than reviews of different problems, and human annotators themselves agree only at $\kappa = 0.74$. Re-joining recovers 24% more duplicate pairs at a cost of five points of precision, and alert groups plus the vague-complaint alarm keep incidents visible when a problem is split.
- Reports, not people. App-store reviews carry no user identity, so counts are reports. Where a company's complaints carry a user ID, counts can be made per user.
- Label source. Play Store labels were made by our team; Bugzilla results rely on independent duplicate links, and the unseen-product labels were cross-checked by LLM labellers.
- LLM dependence. The judge uses an open-weight model through a rate-limited API; all decisions were cached so every reported run is reproducible. Measured on 30 live calls with the deployed prompt and five candidates, a judge decision takes 1.34 s at the median (2.09 s at the 90th percentile), uses about 480 input and 270 output tokens, and costs about $0.00024 at list price, or $0.24 per 1,000 judged complaints; embedding a complaint takes 21 ms on a CPU.
- Deployment without local calibration. Out of the box, about 90% of complaints land in the right case (case precision 0.89–0.92 on an unseen company and unseen products), while pairwise precision there (0.59–0.64) shows that errors

gather in a few large mixed cases. Because every escalation requires human approval and spikes require a burst of reports, such errors surface as a count or a suggestion a reviewer can reject, never as an automatic action. InsightDesk is therefore usable as decision support from the first day; for counts that are trusted without review, a short calibration on the company's own complaints is recommended.

- A single judge model. The judge is one open-weight model (gpt-oss-120b) run at temperature 0 with every decision cached; we did not measure robustness across models or error rates per complaint category, and do not claim its decisions are optimal. One systematic error was observed in a usability test: a complaint describing two problems can join one case, become one of its exemplars and then attract complaints about the other problem. Tracing every join shows this is a minor source of error: joins made through judge-approved exemplars were 85–90% correct and account for 6 of 24 wrong joins on Play Store and 16 of 32 on bug reports, so excluding such exemplars would cost more correct joins than it prevents.
- Assumptions not yet measured. We did not measure how many Bugzilla duplicates are unlinked, so we assume only that the true precision is at least the measured one. We did not test the judge with other models, nor a Bayesian changepoint detector.

VIII. CONCLUSION

InsightDesk turns raw complaints into stable problem cases online and builds incident signals on top of them. A calibrated similarity band lets an LLM judge decide only the uncertain matches, and a re-joining pass merges cases that online decisions split. On human-labelled reviews it reaches precision 0.83, recall 0.35 and F1 0.49, more than doubling the precision of a tuned cross-encoder pipeline (0.38) and outperforming published grouping methods and topic models on the same data. A per-channel lower threshold made short-text grouping both more precise and cheaper, and the same upper threshold transfers without retuning to bug reports (F1 0.78), unseen products and an unseen company. Small-count spike statistics and a vague-complaint alarm make incident detection work at the volumes support desks actually see, and every escalation is approved by a person before it reaches a department.

IX. FUTURE SCOPE

InsightDesk can be packaged with Docker and deployed on a cloud platform, with an open-weight LLM hosted locally for full data privacy and personal details removed from complaints before any LLM call. Corrections that operators already make in the dashboard can be used to calibrate each company automatically. The judge can be tested across models, multi-problem complaints can be kept out of case exemplars, and a Bayesian changepoint detector can be compared with the CUSUM. Per-user submission limits, multilingual complaints and automatic Jira tickets for approved escalations are further extensions.

Acknowledgment

We thank Prof. Ganesh Wadmare for his guidance throughout this project, and the faculty of the Department of Artificial Intelligence & Data Science for their feedback. We also thank Vision Helpdesk, whose support setting shaped the problem studied here.

REFERENCES

[1] J. Dąbrowski, E. Letier, A. Perini and A. Susi, "Analysing app reviews for software engineering: a systematic literature review," Empirical Software Engineering, vol. 27, 2022.

[2] C. Gao et al., “Emerging app issue identification via online joint sentiment-topic tracing,” IEEE Trans. Software Engineering, 2022.

[3] T. Zhang, D. Han, V. Vinayakarao, I. C. Irsan, B. Xu, F. Thung, D. Lo and L. Jiang, “Duplicate bug report detection: How far are we?,” ACM Trans. Software Engineering and Methodology, 2023.

[4] T. Zhang, I. C. Irsan, F. Thung and D. Lo, “Cupid: Leveraging ChatGPT for more accurate duplicate bug report detection,” arXiv:2308.10022, 2023.

[5] R. Peeters, A. Steiner and C. Bizer, “Entity matching using large language models,” in Proc. EDBT, 2025.

[6] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in Proc. NeurIPS (Datasets and Benchmarks), 2023.

[7] J. Gu et al., “A survey on LLM-as-a-judge,” arXiv:2411.15594, 2024.

[8] A. Abbas, K. Tirumala, D. Simig, S. Ganguli and A. S. Morcos, “SemDeDup: Data-efficient learning at web-scale through semantic deduplication,” arXiv:2303.09540, 2023.

[9] N. Muennighoff, N. Tazi, L. Magne and N. Reimers, “MTEB: Massive text embedding benchmark,” in Proc. EACL, 2023.

[10] “iPACK: Incident-aware duplicate ticket aggregation for cloud systems,” in Proc. ICSE, 2023 (arXiv:2302.09520).

[11] F. Liu et al., “TickIt: Leveraging large language models for automated ticket escalation,” in Proc. FSE (industry), 2025.

[12] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang and S. Rajmohan, “Recommending root-cause and mitigation steps for cloud incidents using large language models,” in Proc. ICSE, 2023.

[13] Y. Chen et al., “Automatic root cause analysis via large language models for cloud incidents,” in Proc. EuroSys, 2024.

[14] SentiSum, “What is anomaly detection?,” product documentation, help.sentisum.com, 2025.

[15] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[16] M. Grootendorst, “BERTopic: Neural topic modeling with a class-based TF-IDF procedure,” arXiv:2203.05794, 2022.

[17] D. Loureiro, F. Barbieri, L. Neves, L. Espinosa Anke and J. Camacho-Collados, “TimeLMs: Diachronic language models from Twitter,” in Proc. ACL (demo), 2022.

[18] F. Barbieri, J. Camacho-Collados, L. Espinosa Anke and L. Neves, “TweetEval: Unified benchmark and comparative evaluation for tweet classification,” in Findings of EMNLP, 2020.

[19] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in Proc. NeurIPS, 2020.

[20] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” 2025.

[21] L. Villarroel, G. Bavota, B. Russo, R. Oliveto and M. Di Penta, “Release planning of mobile apps based on user reviews,” in Proc. ICSE, 2016.

[22] C. Sun, D. Lo, S.-C. Khoo and J. Jiang, “Towards more accurate retrieval of duplicate bug reports,” in Proc. ASE, 2011.

[23] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP, 2019.

[24] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL, 2019.

[25] G. S. Manku, A. Jain and A. Das Sarma, “Detecting near-duplicates for web crawling,” in Proc. WWW, 2007.

[26] A. Z. Broder, “On the resemblance and containment of documents,” in Proc. Compression and Complexity of Sequences, 1997.

[27] L. McInnes, J. Healy and J. Melville, “UMAP: Uniform manifold approximation and projection for dimension reduction,” arXiv:1802.03426, 2018.

[28] R. J. G. B. Campello, D. Moulavi and J. Sander, “Density-based clustering based on hierarchical density estimates,” in Proc. PAKDD, 2013.

[29] E. S. Page, “Continuous inspection schemes,” Biometrika, vol. 41, pp. 100–115, 1954.